PACKAGE, ABSTRACT CLASS AND INTERFACE

Objectives

- Package
- Abstract class
- Interface
- Non-access Modifier

PACKAGE, ABSTRACT CLASS AND INTERFACE

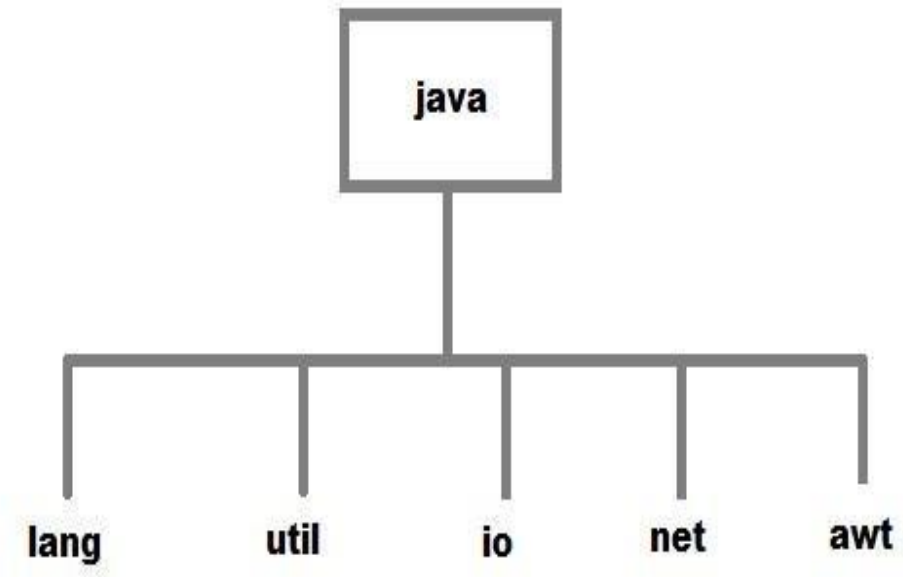
Package

Package are used in Java, in-order to avoid name conflicts and to control access of class, interface and enumeration etc. A package can be defined as a group of similar types of classes, interface, enumeration and sub-package. Using package it becomes easier to locate the related classes.

Package are categorized into two forms

- Built-in Package:-Existing Java package for example java.lang, java.util etc.
- User-defined-package:- Java package created by user to categorized classes and interface

Package are categorized into two forms



Creating a package

Creating a package in java is quite easy. Simply include a package command followed by name of the package as the first statement in java source file.

```
package mypack;
public class employee
{
 ...statement;
}
```

The above statement create a package called **mypack**.

Java uses file system directory to store package. For example the .class for any classes you to define to be part of mypack package must be stored in a directory called mypack

Example of package creation

```
package mypack;
class Book
{
 String bookname;
 String author;
 Book(String b, String c)
 {
  this.bookname = b;
  this.author = c;
 }
 public void show()
```

Abstract class

If a class contain any abstract method then the class is declared as abstract class. An abstract class is never instantiated. It is used to provide abstraction. Although it does not provide 100% abstraction because it can also have concrete method.

Syntax :

```
abstract class class_name { }
```

Abstract method

Method that are declared without any body within an abstract class is known as abstract method. The method body will be defined by its subclass. Abstract method can never be final and static. Any class that extends an abstract class must implement all the abstract methods declared by the super class.

Syntax :

```
abstract return_type function_name ();   // No definition
```

Example of Abstract class

```
abstract class A
{
 abstract void callme();
}
class B extends A
{
 void callme()
 {
  System.out.println("this is callme.");
 }
 public static void main(String[] args)
```

Example of Abstract class

```
 {
  B b=new B();
  b.callme();
 }
}
```

output: this is callme.

Abstract class with concrete(normal) method.

Abstract classes can also have normal methods with definitions, along with abstract methods.

```
abstract class A
{
 abstract void callme();
 public void normal()
 {
  System.out.println("this is concrete method");
 }
}
class B extends A
```

Abstract class with concrete(normal) method.

```
{
 void callme()
 {
  System.out.println("this is callme.");
 }
 public static void main(String[] args)
 {
  B b=new B();
  b.callme();
  b.normal();
 }
}
```

output:

this is callme.

this is concrete method.

When to use Abstract Methods & Abstract Class?

Abstract methods are usually declared where two or more subclasses are expected to do a similar thing in different ways through different implementations. These subclasses extend the same Abstract class and provide different implementations for the abstract methods.

Abstract classes are used to define generic types of behaviors at the top of an object-oriented programming class hierarchy, and use its subclasses to provide implementation details of the abstract class.

Interface

Interface is a pure abstract class.They are syntactically similar to classes, but you cannot create instance of an **Interface** and their methods are declared without any body. Interface is used to achieve complete **abstraction** in Java. When you create an interface it defines what a class can do without saying anything about how the class will do it.

Syntax :

```
interface interface_name { }
```

Example of Interface

```
interface Moveable
{
 int AVERAGE-SPEED=40;
 void move();
}
```

Example of Interface

```
interface Moveable
int AVERAGE-SPEED=40;
void move();
```

what you declare

```
interface Moveable
public static final int AVERAGE-SPEED=40;
public abstract void move();
```

what the compiler sees

NOTE : Compiler automatically converts methods of Interface as public and abstract, and the data members as public, static and final by default.

Rules for using Interface

- Methods inside Interface must not be static, final, native or strict.
- All variables declared inside interface are implicitly public static final variables (constants).
- All methods declared inside Java Interfaces are implicitly public and abstract, even if you don't use public or abstract keyword.
- Interface can extend one or more other interface.
- Interface cannot implement a class.
- Interface can be nested inside another interface.

Example of Interface implementation

```
interface Moveable
{
 int AVG-SPEED = 40;
 void move();
}

class Vehicle implements Moveable
{
 public void move()
 {
  System .out. print in ("Average speed is"+AVG-SPEED");
```

Example of Interface implementation

```
 }
 public static void main (String[] arg)
 {
  Vehicle vc = new Vehicle();
  vc.move();
 }
}
```

Output:

Average speed is 40.

Interfaces supports Multiple Inheritance

Though a class in java doesn't support multiple inheritances, but a class can implement more than one interface.

```
interface Moveable
{
 boolean isMoveable();
}

interface Rollable
{
 boolean isRollable
}
```

Interfaces supports Multiple Inheritance

```
class Tyre implements Moveable, Rollable
{
 int width;

 boolean isMoveable()
 {
  return true;
 }

 boolean isRollable()
 {
```

Interfaces supports Multiple Inheritance

```
 return true;
 }
 public static void main(String args[])
 {
  Tyre tr=new Tyre();
  System.out.println(tr.isMoveable());
  System.out.println(tr.isRollable());
 }
}
```

Output:

```
true
true
```

Interface extends other Interface

Classe implements interfaces, but an interface extends other interface.

```
interface NewsPaper
{
 news();
}

interface Magazine extends NewsPaper
{
 colorful();
}
```

Difference between an interface and an abstract class?

Abstract class	Interface
Abstract class is a class which contains one or more abstract methods, which has to be implemented by its sub classes.	Interface is a Java Object containing method declaration but no implementation. The classes which implement the Interfaces must provide the method definition for all the methods.
Abstract class is a Class prefix with an abstract keyword followed by Class	Interface is a pure abstract class which starts with interface keyword.

Difference between an interface and an abstract class?

Abstract class	Interface
definition.	
Abstract class can also contain concrete methods.	Whereas, Interface contains all abstract methods and final variable declarations.
Abstract classes are useful in a situation that Some general methods should be implemented and specialization behavior should be implemented by child classes.	Interfaces are useful in a situation that all properties should be implemented.

Final

Final Variables:

A final variable can be explicitly initialized only once. A reference variable declared final can never be reassigned to refer to an different object.

However the data within the object can be changed. So the state of the object can be changed but not the reference.

With variables, the *final* modifier often is used with *static* to make the constant a class variable.

Final Variables:

Example:

```
publicclassTest{
final int value =10;
// The following are examples of declaring constants:
public static final int BOXWIDTH =6;  static final
String TITLE ="Manager";

publicvoid changeValue(){
value =12;//will give an error
}
}
```

Final Methods:

A final method cannot be overridden by any subclasses. As mentioned previously the final modifier prevents a method from being modified in a subclass.

The main intention of making a method final would be that the content of the method should not be changed by any outsider.

Example:

You declare methods using the *final* modifier in the class declaration, as in the following example:

```
public class Test{
public final void changeName(){
// body of method
}
}
```

Final Classes:

The main purpose of using a class being declared as *final* is to prevent the class from being sub classed. If a class is marked as final then no class can inherit any feature from the final class.

Example:

```
Public final class Test{
// body of class
}
```

Summary

- Abstract class/method
- Interface
- Final class/method/variable